SQL-PROBLEMS

1.Afișează firstName, lastName, city și state pentru **toate persoanele**. Dacă o persoană nu are adresă, afișează NULL.

Person(personId, firstName, lastName)
Address(addressId, personId, city, state)

```
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a ON a.personId = p.personId;
```

2.Găsește **al TREILEA cel mai mare salariu**. Dacă nu există, returnează NULL.

–prima varianta –
(In SQL, cand aplici o functie de agregare precum `MAX()` pe o multime vida (un tabel cu zero randuri), baza de date genereaza si returneaza automat valoarea `NULL`. Nu da eroare si nu iti da un rezultat "invizibil", ci pur si simplu zice: "Maximul din nimic este NULL".)

–al 2 lea cel mai mare salariu–

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employee);
```

–a 2 varianta –
(Daca interogarea din paranteze este goala, acel `SELECT` exterior o transforma automat in `NULL`)

```
SELECT (
SELECT DISTINCT(salary)
FROM employees
ORDER BY salary desc
LIMIT 1 OFFSET 2
)
AS ThirdHighestSalary;
```

–a 3 a varianta –
(Am folosit `MAX(salary)` in selectul final ca un mic truc tocmai pentru a forta baza de date sa returneze NULL in caz ca `rank_num = 3` nu exista deloc in rezultate.)
(Este o functie care imparte "medalii" (locul 1, locul 2, etc.) pentru fiecare salariu. //Daca Ana are 5000 lei si Bogdan are tot 5000 lei, ambii primesc rank-ul 1.)
WITH (Clauza `WITH` se mai numeste si CTE (Common Table Expression). Gandeste-te la ea ca la o variabila temporara sau un tabel virtual pe care il creezi in memorie doar cat timp ruleaza interogarea.
"Creeaza tu intai tabelul asta cu tot cu clasament si tine-l putin in memorie. Dupa ce e gata, noi facem un SELECT separat din el si abia atunci putem sa folosim `WHERE rank_num = 3`".)

```
WITH nume_tabela AS
(
SELECT
salary , DENSE_RANK() OVER (ORDER  BY salary DESC ) as rank_number
)

SELECT MAX(salary)
FROM nume_tabela
WHERE rank_number=3
```

3.Găsește **al N-lea cel mai mare salariu**. Dacă nu există, returnează NULL.

```
CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
BEGIN
 RETURN (
    # Write your MySQL query statement below.
      WITH nume_tabela as
      (
         SELECT
         salary, DENSE_RANK() OVER(order by salary desc ) as nr_rank
         FROM employee
      )
      SELEct MAx(salary) as getNthHighestSalary
      from nume_tabela
      where nr_rank=N
```

```
 );
END
```

4.Afișează fiecare scor cu **rankul** lui. Scoruri egale primesc același rank, fără a sări numere.

```
SELECT
  score,
  DENSE_RANK() OVER (ORDER BY score DESC) AS 'rank'
FROM scores
```

5.Găsește toate numerele care apar de cel puțin 3 ori consecutiv în tabel.

(LEAD(num, 1) OVER (ORDER BY id): Ii spune bazei de date: "Aseaza randurile frumos in ordine dupa ID. Acum, pentru randul la care te afli, trage cu ochiul la numarul de pe urmatorul rand (1 pas in fata) si copiaza-l in coloana noua numita next_1.")

```
WITH NextNumbers AS (
   SELECT
      num,
      LEAD(num, 1) OVER (ORDER BY id) AS next_1, -- se uita la numarul de
pe randul urmator
      LEAD(num, 2) OVER (ORDER BY id) AS next_2  -- se uita la numarul de
peste 2 randuri
   FROM Logs
)
SELECT DISTINCT num AS ConsecutiveNums
FROM NextNumbers
WHERE num = next_1 AND num = next_2;
```

–varianta 2-

```
# Write your MySQL query statement below
SELECT DIstinct l1.num  as ConsecutiveNums
FROM Logs l1,Logs l2, Logs l3
where l1.id=l2.id-1 -- l2 este fix randul urmator dupa l1
AND l2.id=l3.id-1
and l1.num=l2.num
and l2.num=l3.num
```

6.Afișează numele angajaților care câștigă mai mult decât managerul lor direct.

```
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m ON e.managerId = m.id
WHERE e.salary > m.salary;
```

7.Găsește toate emailurile duplicate.

```
SELECT email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

8.Afișează numele clienților care nu au plasat nicio comandă.

-varianta 1-

```
SELECT name AS Customers
FROM Customers
WHERE id NOT IN (SELECT customerId FROM Orders);
```

—varianta 2–

(Daca schimbi in LEFT JOIN, baza de date va aduce absolut toti clientii pe ecran. La cei care au cumparat ceva, va pune datele comenzii in dreapta. La cei care nu au cumparat nimic, va pune automat NULL.)

```
SELECT c.name AS Customers
FROM Customers c
LEFT JOIN Orders o ON c.id = o.customerId
WHERE o.customerId IS NULL;
```

9.Afișează departamentul, numele angajatului și salariul pentru angajatul cu cel mai mare salariu din fiecare departament.

```
—-varianta 1—-
SELECT d.name AS Department, e.name AS Employee, e.salary
FROM Employee e
JOIN Department d ON e.departmentId = d.id
WHERE (e.departmentId, e.salary) IN (
   SELECT departmentId, MAX(salary)
   FROM Employee
   GROUP BY departmentId
);

—-varianta 2—-
with nume_tabela as
(
Select d.name AS Department, e.name AS Employee, e.salary,
DENSE_RANK() over(PARTITION BY e.departmentId ORDER BY e.salary
desc) as rank_per_derpartament
FROM Employee e
JOIN Department d ON e.departmentId = d.id
)


SELECT Department, Employee, Salary
FROM nume_tabela
WHERE rank_per_derpartament = 1;
```

10.Schimbă locurile între fiecare pereche de vecini. Ultimul rămâne pe loc dacă numărul e impar

```
—-nu imi schimba randul la case doar ii da randului respectiv un id mai mic
schimbarea se intampla la order by id –
# Write your MySQL query statement below
Select
case
      When (id % 2 = 1 and (id+1 <= (select max(id) from seat))) Then id+1
      when id%2 = 0 then id-1
      else id
END as id,
```

student
from seat
order by id

new_id	student
2(avea 1dar face +1)	Ana
1	Bogdan
4	Carmen
3	Dan

11.Într-un singur UPDATE, schimbă toate valorile din sex: 'm' → 'f' și 'f' → 'm'.
Salary(id, name, sex, salary)

```
UPDATE Salary
SET sex = CASE
   WHEN sex = 'm' THEN 'f'
   ELSE 'm'
END;
```

12.Scrie o singura interogare cu UPDATE care sa modifice coloana salariu_text respectand urmatoarele conditii:
Daca valoarea matematica a salariului este strict mai mica decat 5000, schimba-l in '6500' (ca text).
Daca valoarea matematica a salariului este intre 6000 si 7000 (inclusiv), schimba-l in '4500' (ca text).
Restul salariilor trebuie sa ramana exact asa cum sunt.

```
UPDATE Contracte
SET salariu_text = CASE
     WHEN CAST(salariu_text AS SIGNED) < 5000 THEN '6500'
   WHEN CAST(salariu_text AS SIGNED) BETWEEN 6000 AND 7000 THEN
'4500'
   ELSE salariu_text
```

END;

Din INTEGER in TEXT: TO_CHAR(coloana). Partea geniala la TO_CHAR e ca poti sa o si formatezi in acelasi timp. De exemplu, daca scrii TO_CHAR(salariu, '$99,999'), iti ia numarul 5000 si ti-l transforma direct in textul frumos formatat "$5,000".
Din TEXT in INTEGER: TO_NUMBER(coloana).

13.Concateneaza mi la toate salariile mai mici de cât 5000 semnul $.

```
SELECT
   nume,
   CASE
      WHEN CAST(salariu_text AS SIGNED) < 5000
 THEN CONCAT('$', salariu)
      ELSE CAST(salariu AS CHAR)
   END AS salariu_formatat
FROM Angajati;
```

—--UPDATE—--

```
UPDATE Contracte
SET salariu_text = CASE
   WHEN CAST(salariu_text AS SIGNED) < 5000 THEN CONCAT('$',
salariu_text)
   ELSE salariu_text
END;
```

14.570. Managers with at Least 5 Direct Reports

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| name        | varchar |
| department  | varchar |
| managerId   | int     |
+-------------+---------+
```

id is the primary key (column with unique values) for this table.
Each row of this table indicates the name of an employee, their department, and the id of their manager.
If managerId is null, then the employee does not have a manager.
No employee will be the manager of themself.

Write a solution to find managers with at least **five direct reports**.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:
Employee table:
```
+-----+-------+------------+-----------+
| id  | name  | department | managerId |
+-----+-------+------------+-----------+
| 101 | John  | A          | null      |
| 102 | Dan   | A          | 101       |
| 103 | James | A          | 101       |
| 104 | Amy   | A          | 101       |
| 105 | Anne  | A          | 101       |
| 106 | Ron   | B          | 101       |
+-----+-------+------------+-----------+
```
Output:
```
+------+
| name |
+------+
| John |
+------+
```
//
```
SELECT m.name
FROM employee e
JOIN employee m ON e.managerId = m.id
GROUP BY m.id
HAVING COUNT(e.id) >= 5;
```

15. 1934. Confirmation Rate

The **confirmation rate** of a user is the number of 'confirmed' messages divided by the total number of requested confirmation messages. The confirmation rate of a user that did not request any confirmation messages is 0. Round the confirmation rate to **two decimal** places.

Write a solution to find the **confirmation rate** of each user.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Signups table:

```
+---------+---------------------+
| user_id | time_stamp          |
+---------+---------------------+
| 3       | 2020-03-21 10:16:13 |
| 7       | 2020-01-04 13:57:59 |
| 2       | 2020-07-29 23:09:44 |
| 6       | 2020-12-09 10:39:37 |
+---------+---------------------+
```

Confirmations table:

```
+---------+---------------------+-----------+
| user_id | time_stamp          | action    |
+---------+---------------------+-----------+
| 3       | 2021-01-06 03:30:46 | timeout   |
| 3       | 2021-07-14 14:00:00 | timeout   |
| 7       | 2021-06-12 11:57:29 | confirmed |
| 7       | 2021-06-13 12:58:28 | confirmed |
| 7       | 2021-06-14 13:59:27 | confirmed |
| 2       | 2021-01-22 00:00:00 | confirmed |
| 2       | 2021-02-28 23:59:59 | timeout   |
+---------+---------------------+-----------+
```

Output:

```
+---------+-------------------+
| user_id | confirmation_rate |
+---------+-------------------+
| 6       | 0.00              |
| 3       | 0.00              |
| 7       | 1.00              |
| 2       | 0.50              |
+---------+-------------------+
```

```
# Write your MySQL query statement below
select s.user_id,
ROUND(AVG(case when c.action='confirmed' then 1 else 0 END),2) as
confirmation_rate
from signups s
left join confirmations c on c.user_id=s.user_id
group by s.user_id
```

16. 1193. Monthly Transactions I

Write an SQL query to find for each month and country, the number of transactions and their total amount, the number of approved transactions and their total amount.

Return the result table in **any order**.

The query result format is in the following example.

Example 1:

Input:

Transactions table:

```
+------+---------+----------+--------+------------+
| id   | country | state    | amount | trans_date |
+------+---------+----------+--------+------------+
| 121  | US      | approved | 1000   | 2018-12-18 |
| 122  | US      | declined | 2000   | 2018-12-19 |
| 123  | US      | approved | 2000   | 2019-01-01 |
| 124  | DE      | approved | 2000   | 2019-01-07 |
+------+---------+----------+--------+------------+
```

Output:

```
+----------+---------+-------------+----------------+-------------------+-----------------------+
| month    | country | trans_count | approved_count | trans_total_amount | approved_total_amount |
+----------+---------+-------------+----------------+-------------------+-----------------------+
| 2018-12  | US      | 2           | 1              | 3000              | 1000                  |
| 2019-01  | US      | 1           | 1              | 2000              | 2000                  |
| 2019-01  | DE      | 1           | 1              | 2000              | 2000                  |
+----------+---------+-------------+----------------+-------------------+-----------------------+
```

```
select DATE_FORMAT(trans_date,'%Y-%m') as month ,country ,
count(*) as trans_count,
sum(amount) as trans_total_amount,
sum(case when state='approved' then 1 else 0 end ) as approved_count,
sum(case when state = 'approved' then amount else 0 end ) as
approved_total_amount
from transactions
group by month,country
```

17.1174. Immediate Food Delivery II

If the customer's preferred delivery date is the same as the order date, then the order is called **immediate;** otherwise, it is called **scheduled**.

The **first order** of a customer is the order with the earliest order date that the customer made. It is guaranteed that a customer has precisely one first order.

Write a solution to find the percentage of immediate orders in the first orders of all customers, **rounded to 2 decimal places**.

The result format is in the following example.

Example 1:

Input:

Delivery table:

```
+-------------+-------------+------------+----------------------------+
| delivery_id | customer_id | order_date | customer_pref_delivery_date |
+-------------+-------------+------------+----------------------------+
| 1           | 1           | 2019-08-01 | 2019-08-02                 |
| 2           | 2           | 2019-08-02 | 2019-08-02                 |
| 3           | 1           | 2019-08-11 | 2019-08-12                 |
| 4           | 3           | 2019-08-24 | 2019-08-24                 |
| 5           | 3           | 2019-08-21 | 2019-08-22                 |
| 6           | 2           | 2019-08-11 | 2019-08-13                 |
| 7           | 4           | 2019-08-09 | 2019-08-09                 |
+-------------+-------------+------------+----------------------------+
```

Output:

```
+---------------------+
| immediate_percentage |
+---------------------+
| 50.00               |
+---------------------+
```

Explanation:

The customer id 1 has a first order with delivery id 1 and it is scheduled.

The customer id 2 has a first order with delivery id 2 and it is immediate.

The customer id 3 has a first order with delivery id 5 and it is scheduled.

The customer id 4 has a first order with delivery id 7 and it is immediate.

Hence, half the customers have immediate first orders.

```
# Write your MySQL query statement below
select
   ROUND(AVG(case when order_date=customer_pref_delivery_date then 1 else 0
end )*100,2)AS immediate_percentage
from delivery
where (customer_id,order_date)  in
(select
   customer_id,
   min(order_date)
   from delivery
   group by customer_id
)
```

18.550. Game Play Analysis IV

Write a solution to report the **fraction** of players that logged in again on the day after the day they first logged in, **rounded to 2 decimal places**. In other words, you need to determine the number of players who logged in on the day immediately following their initial login, and divide it by the number of total players.

The result format is in the following example.

Example 1:

Input:

Activity table:

```
+-----------+-----------+------------+--------------+
| player_id | device_id | event_date | games_played |
+-----------+-----------+------------+--------------+
| 1         | 2         | 2016-03-01 | 5            |
| 1         | 2         | 2016-03-02 | 6            |
| 2         | 3         | 2017-06-25 | 1            |
| 3         | 1         | 2016-03-02 | 0            |
| 3         | 4         | 2018-07-03 | 5            |
+-----------+-----------+------------+--------------+
```

Output:

```
+-----------+
| fraction  |
+-----------+
| 0.33      |
+-----------+
```

Explanation:

Only the player with id 1 logged back in after the first day he had logged in so the answer is 1/3 = 0.33

```
# Write your MySQL query statement below
select
round(count(player_id)/(select count(Distinct(player_id)) from activity) ,2 )
as fraction
from activity
where (player_id , event_date) IN
(
   select
   player_id,
   date_add(min(event_date), interval 1 day)
   from activity
   group by player_id
)
```

19. 1070. Product Sales Analysis III

Write a solution to find all sales that occurred in the **first year** each product was sold.

- For each product_id, identify the earliest year it appears in the Sales table.
- Return **all** sales entries for that product in that year.

Return a table with the following columns: **product_id**, **first_year**, **quantity,** and **price**. Return the result in any order.

Example 1:

Input:

Sales table:

```
+---------+------------+------+----------+-------+
| sale_id | product_id | year | quantity | price |
+---------+------------+------+----------+-------+
| 1       | 100        | 2008 | 10       | 5000  |
| 2       | 100        | 2009 | 12       | 5000  |
| 7       | 200        | 2011 | 15       | 9000  |
+---------+------------+------+----------+-------+
```

Output:

```
+------------+------------+----------+-------+
| product_id | first_year | quantity | price |
+------------+------------+----------+-------+
| 100        | 2008       | 10       | 5000  |
| 200        | 2011       | 15       | 9000  |
+------------+------------+----------+-------+
```

```
# Write your MySQL query statement below
select product_id, year as first_year,quantity,price
from sales
where (product_id,year) IN
(   select product_id,MIN(year)
   from sales
   group by product_id
)
```

19. 1045. Customers Who Bought All Products

Table: Product

```
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| product_key | int     |
+-------------+---------+
```

product_key is the primary key (column with unique values) for this table.

Write a solution to report the customer ids from the Customer table that bought all the products in the Product table.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Customer table:

```
+-------------+-------------+
| customer_id | product_key |
+-------------+-------------+
| 1           | 5           |
| 2           | 6           |
| 3           | 5           |
| 3           | 6           |
| 1           | 6           |
+-------------+-------------+
```

Product table:

```
+-------------+
| product_key |
+-------------+
```

| 5 |
| 6 |
+----------+

Output:

+----------+
| customer_id |
+----------+
| 1 |
| 3 |
+----------+

Explanation:

The customers who bought all the products (5 and 6) are customers with IDs 1 and 3.

```
# Write your MySQL query statement below
select
c.customer_id
from customer c
group by customer_id
having count(DISTINCT product_key) = (SELECT count(*) from product)
```

20. 1164. Product Price at a Given Date

Initially, all products have price 10.

Write a solution to find the prices of all products on the date 2019-08-16.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Products table:

+----------+----------+----------+
| product_id | new_price | change_date |
+----------+----------+----------+
| 1 | 20 | 2019-08-14 |
| 2 | 50 | 2019-08-14 |

1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17
3	20	2019-08-18

Output:

product_id	price
2	50
1	35
3	10

```
# Write your MySQL query statement below
(select product_id,new_price as price
from products
where (product_id,change_date) in
(select product_id,max(change_date)
from products
where change_date<="2019-08-16"
group by product_id))

union
#verificam pentru cei care au schim
(select DISTINCT product_id ,10 as price
from products
where product_id not in
(select product_id
from products
where change_date<="2019-08-16"
)
)
```

21. 1204. Last Person to Fit in the Bus

There is a queue of people waiting to board a bus. However, the bus has a weight limit of 1000 **kilograms**, so there may be some people who cannot board.

Write a solution to find the person_name of the **last person** that can fit on the bus without exceeding the weight limit. The test cases are generated such that the first person does not exceed the weight limit.

Note that *only one* person can board the bus at any given turn.

The result format is in the following example.

Example 1:

Input:
Queue table:

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

Output:

person_name
John Cena

Explanation: The folowing table is ordered by the turn for simplicity.

Turn	ID	Name	Weight	Total Weight	
1	5	Alice	250	250	
2	3	Alex	350	600	
3	6	John Cena	400	1000	(last person to board)
4	2	Marie	200	1200	(cannot board)
5	4	Bob	175	___	
6	1	Winston	500	___	

```
+------+----+-----------+--------+-------------+
```

```
# Write your MySQL query statement below
select person_name
from
(
   select
   person_name,turn,
   SUM(weight) over (order by turn) as total_weight
   from queue
) tabel_total
where total_weight<=1000
order by turn desc
limit 1;
```